ChatBot

Atividade do Negócio

Autor: Bruno Salles
Mogi Das Cruzes, 20 de Junho de 2024

Sumário

1 **Introdução** ... 3
2 **Objetivo** ... 3
3 **Fluxo de Atividade** ... 3
3.1 Configuração do Banco de Dados ... 3
3.1.1 *Objetivo* ... 3
3.1.2 *Componentes:* ... 3
3.1.3 *Código*: ... 4
3.2 Salvar Status do Envio ... 4
3.2.1 *Objetivo:* ... 4
3.2.2 *Componentes:* ... 4
3.2.3 *Código*: ... 4
3.3 Consulta de Dados ... 5
3.3.1 *Objetivo* ... 5
3.3.2 *Componentes:* ... 5
3.3.3 *Código*: ... 5
3.4 Envio de Mensagens ... 6
3.4.1 *Objetivo* ... 6
3.4.2 Componente: ... 6
3.4.3 *Código:* ... 7
3.5 Seleção de Arquivo Excel ... 7
3.5.1 *Objetivo* ... 7
3.5.2 *Componente*: ... 7
3.5.3 *Código:* ... 8
3.6 Execução do envio de mensagens ... 8
3.6.1 *Objetivo* ... 8
3.6.2 *Componente*: ... 8
3.6.3 *Código:* ... 8
3.7 Interface Gráfica ... 8
3.7.1 *Objetivo* ... 8
3.7.2 *Componente*: ... 8
3.7.3 *Código:* ... 9
4 **Conclusão** ... 9

1 Introdução

Este documento detalha as atividades de negócios para o uso de um sistema automatizado de envio de mensagens via WhatsApp, utilizando um arquivo Excel como fonte de dados. O sistema também permite a consulta do status dos envios através de uma interface gráfica amigável. O objetivo é fornecer uma visão clara e estruturada das etapas necessárias para operar o sistema, garantindo uma utilização eficiente e eficaz.

2 Objetivo

Esse sistema é destinado a automatizar o processo de envio de mensagens via WhatsApp, proporcionando um meio eficiente de comunicação em massa, e possibilitar a consulta dos envios realizados.

3 Fluxo de Atividade

3.1 Configuração do Banco de Dados

3.1.1 *Objetivo*

- Configurar o banco de dados SQLite para armazenar informações sobre os envios de mensagens.

3.1.2 *Componentes:*

- Função configurar_bd()

3.1.3 *Código*:

```python
def configurar_bd():

    conn = sqlite3.connect('envios.db')

    cursor = conn.cursor()

    cursor.execute('''

    CREATE TABLE IF NOT EXISTS envios (

        id INTEGER PRIMARY KEY AUTOINCREMENT,

        telefone TEXT,

        status TEXT,

        mensagem TEXT,

        data_hora TEXT

    )

    ''')

    conn.commit()

    cursor.execute("PRAGMA table_info(envios);")

    columns = [info[1] for info in cursor.fetchall()]

    if 'data_hora' not in columns:

        cursor.execute('ALTER TABLE envios ADD COLUMN data_hora TEXT;')

    conn.commit()

    conn.close()
```

3.2 Salvar Status do Envio

3.2.1 *Objetivo:*

- Registrar o status de cada envio no banco de dados.

3.2.2 *Componentes:*

- Função salvar_status(telefone, status, mensagem):

3.2.3 *Código*:

```python
def salvar_status(telefone, status, mensagem):

    conn = sqlite3.connect('envios.db')

    cursor = conn.cursor()

    data_hora = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    cursor.execute('''

    INSERT INTO envios (telefone, status, mensagem, data_hora)

    VALUES (?, ?, ?, ?)

    ''', (telefone, status, mensagem, data_hora))

    conn.commit()

    conn.close()
```

3.3 Consulta de Dados

3.3.1 *Objetivo*

- Consultar o status dos envios realizados com base em filtros especificados

3.3.2 *Componentes:*

- Função consultar_dados()

3.3.3 *Código*:

```python
def consultar_dados():
    conn = sqlite3.connect('envios.db')

    cursor = conn.cursor()

    contato = entrada_contato.get().strip()

    status = entrada_status.get().strip()

    data_inicio = entrada_data_inicio.get().strip()

    data_fim = entrada_data_fim.get().strip()

    query = "SELECT * FROM envios WHERE 1=1"

    if contato:

        query += f" AND telefone LIKE '%{contato}%'"

    if status:

        query += f" AND status LIKE '%{status}%'"

    if data_inicio:

        query += f" AND data_hora >= '{data_inicio} 00:00:00'"

    if data_fim:

        query += f" AND data_hora <= '{data_fim} 23:59:59'"

    cursor.execute(query)

    rows = cursor.fetchall()

    conn.close()


    for item in tree.get_children():

        tree.delete(item)

    for row in rows:

        tree.insert("", "end", text=row[0], values=(row[1], row[2], row[3], row[4]))
```

3.4 Exportar Dados

3.4.1 *Objetivo*

- Exportar dados históricos do banco de dados.

3.4.2 *Componentes:*

- Função exportar_dados()

3.4.3 *Código*:

```
def exportar_dados():

    conn = sqlite3.connect('envios.db')

    df = pd.read_sql_query("SELECT * FROM envios", conn)

    conn.close()

    file_path = filedialog.asksaveasfilename(defaultextension=".xlsx", filetypes=[("Excel files", "*.xlsx")])

    if file_path:

        df.to_excel(file_path, index=False)

        messagebox.showinfo("Informação", "Dados exportados com sucesso.")

```

3.5 Envio de Mensagens

3.5.1 *Objetivo*

- Enviar mensagens para os contatos listados no Excel.

3.5.2 Componente:

- Função enviar_mensagens(file_path).

3.5.3 *Código:*

```python
def enviar_mensagens(file_path):

    df = pd.read_excel(file_path)
    coluna_contato = 'Contato'
    coluna_mensagens = [f'Mensagem{i}' for i in range(1, 8)]
    codigo_pais = '+55'

    for index, row in df.iterrows():

        telefone_destinatario = str(row[coluna_contato]).strip()

        if not telefone_destinatario.startswith('+'):

            telefone_destinatario = codigo_pais + telefone_destinatario

        mensagens = '\n'.join([str(row[col]).strip() if not pd.isna(row[col]) else '' for col in coluna_mensagens])

        if pd.isna(telefone_destinatario) or telefone_destinatario.strip() == '':

            messagebox.showerror(title=None, message="Telefone vazio encontrado, parando o envio.")

            break

        try:

            kit.sendwhatmsg_instantly(telefone_destinatario, mensagens)

            time.sleep(2)

            screenshot = p.screenshot()

            screenshot.save("img/screenshot.png")

            error_location = p.locateOnScreen("img/ERROR.png", confidence=0.8)

            if error_location:

                error_message = "O número de telefone compartilhado por URL é inválido."

                salvar_status(telefone_destinatario, error_message, "Mensagem não enviada")

                print("Mensagem não enviada",telefone_destinatario)

                p.hotkey('ctrl', 'shift', 'tab')

                p.hotkey('ctrl', 'w')
```

```python
        except Exception as e:

            salvar_status(telefone_destinatario, f"Enviado {str(e)}", mensagens)

            print("Mensagem enviada",telefone_destinatario)

            p.hotkey('ctrl', 'shift', 'tab')

            p.hotkey('ctrl', 'w')

    messagebox.showinfo(title='ChatBot', message='Mensagens enviadas com sucesso!')
```

3.6 Seleção de Arquivo Excel

3.6.1 *Objetivo*

- Selecionar o arquivo Excel contendo os contatos e as mensagens.

3.6.2 *Componente*:

- Função selecionar_arquivo()

3.6.3 *Código:*

```python
def selecionar_arquivo():

    file_path = filedialog.askopenfilename(title="Selecione o arquivo Excel", filetypes=[("Excel files", "*.xlsx *.xls")])

    entrada_var.set(file_path)
```

3.7 Execução do envio de mensagens

3.7.1 *Objetivo*

- Executar o processo de envio de mensagens.

3.7.2 *Componente*:

- Função executar()

3.7.3 *Código:*

```python
def executar():

    file_path = entrada_var.get()

    if file_path:

        enviar_mensagens(file_path)

    else:

        messagebox.showerror(title=None, message="Nenhum arquivo selecionado.")
```

3.8 Interface Gráfica

3.8.1 *Objetivo*

- Fornecer uma interface amigável para interação do usuário.

3.8.2 *Componente*:

- Função iniciar_interface()

3.8.3 *Código:*

```python
def iniciar_interface():

    global entrada_var, entrada_contato, entrada_status, entrada_data_inicio, entrada_data_fim, tree

    root = tk.Tk()

    root.title("Processador de Arquivos Excel")

    tk.Label(root, text="Selecione o arquivo de entrada:").grid(row=0, column=0, padx=10, pady=10)

    entrada_var = tk.StringVar()

    tk.Entry(root, textvariable=entrada_var, width=50).grid(row=0, column=1, padx=10, pady=10)

    tk.Button(root, text="Procurar", command=selecionar_arquivo).grid(row=0, column=2, padx=10, pady=10)

    tk.Label(root, text="Filtro por Contato:").grid(row=1, column=0, padx=10, pady=10)

    entrada_contato = tk.Entry(root, width=30)

    entrada_contato.grid(row=1, column=1, padx=10, pady=10)

    tk.Label(root, text="Filtro por Status:").grid(row=2, column=0, padx=10, pady=10)

    entrada_status = tk.Entry(root, width=30)

    entrada_status.grid(row=2, column=1, padx=10, pady=10)

    tk.Label(root, text="Filtro por Data Início (YYYY-MM-DD):").grid(row=3, column=0, padx=10, pady=10)

    entrada_data_inicio = tk.Entry(root, width=30)

    entrada_data_inicio.grid(row=3, column=1, padx=10, pady=10)

    tk.Label(root, text="Filtro por Data Fim (YYYY-MM-DD):").grid(row=4, column=0, padx=10, pady=10)

    entrada_data_fim = tk.Entry(root, width=30)

    entrada_data_fim.grid(row=4, column=1, padx=10, pady=10)

    tk.Button(root, text="Executar", command=executar).grid(row=5, column=0, columnspan=3, pady=20)

    tk.Button(root, text="Consultar", command=consultar_dados).grid(row=6, column=0, columnspan=3, pady=20)
```

```python
    tree = ttk.Treeview(root, columns=("Telefone", "Status", "Mensagem", "Data/Hora"), show="headings")

    tree.heading('Telefone', text='Telefone')

    tree.heading('Status', text='Status')

    tree.heading('Mensagem', text='Mensagem')

    tree.heading('Data/Hora', text='Data/Hora')

    tree.grid(row=7, column=0, columnspan=3, padx=10, pady=10, sticky="nsew")

    root.mainloop()

if __name__ == "__main__":

    configurar_bd()

    iniciar_interface()
```

4 Conclusão

Este documento fornece uma visão detalhada das atividades de negócios associadas ao sistema de envio automatizado de mensagens via WhatsApp. Seguindo as etapas descritas, os usuários podem configurar o banco de dados, selecionar arquivos Excel, enviar mensagens e consultar o status dos envios. A interface gráfica amigável facilita a interação do usuário, tornando o sistema eficiente e acessível para comunicação em massa.